A+ Plane

Frontend Vibe-Coding Prompt Book

A structured collection of English prompts for AI-assisted development of the A+ Plane college grievance portal frontend.

Project	A+ Plane — College Grievance and Campus Support Portal
Frontend stack	React, Vite, React Router, Tailwind CSS, Axios, Lucide React
Development approach	Iterative AI-assisted / vibe coding: describe requirements, generate or edit code, run the application, review errors, and refine.
Purpose	To document the prompts that can guide the frontend build, individual screens, chatbot improvements, integration, and quality checks.

Documentation note: This is an organized, reusable prompt pack based on the A+ Plane requirements and frontend structure discussed for the project. It is not a verbatim transcript of every earlier chat message, and it does not by itself prove that every listed feature has been implemented or tested.

Prepared for project development and demonstration

1. How to Use This Prompt Book

Use these prompts with an AI coding assistant such as Antigravity or Claude Code while the existing A+ Plane project folder is open. Start with the master prompt, then use the relevant page-level prompts and the integration prompt. Ask the assistant to edit the existing files instead of creating duplicate implementations.

Recommended vibe-coding workflow

- **Describe:** explain the user problem and the expected screen or interaction.
- **Inspect:** ask the AI to read the existing project structure and current code before editing.
- **Generate or refine:** request focused changes to the existing React components and services.
- **Run:** start the development server and inspect the page in a browser.
- **Debug:** share actual errors and ask for targeted fixes rather than a complete rewrite.
- **Verify:** check navigation, forms, loading/error states, responsiveness, and API behavior.
- **Document:** record which prompts were used, which files changed, and which checks actually passed.

Frontend files in scope

The project structure discussed includes the following files and modules. The AI assistant must confirm the actual files in the workspace before making changes.

```
frontend/
  src/
    components/
      AIChatbot.jsx
      ComplaintCard.jsx
      Navbar.jsx
      ProtectedRoute.jsx
      Sidebar.jsx
      StatusBadge.jsx
    pages/
      Login.jsx
      StudentDashboard.jsx
      RaiseGrievance.jsx
      MyComplaints.jsx
      ComplaintDetails.jsx
      AdminDashboard.jsx
      NotFound.jsx
    modules/
      bus/BusForm.jsx
      hostel/HostelForm.jsx
      hostel/hostelCategories.js
      subjects/SubjectForm.jsx
      her-voice/HerVoiceForm.jsx
      lost-found/LostFoundForm.jsx
    services/
      api.js
      auth.js
    App.jsx
    main.jsx
```

The list above reflects the project structure discussed and may not include every file currently present in the repository.

2. Master Prompt — Build the Existing Frontend

Use this first to ask the AI coding assistant to inspect the existing frontend and make coherent, incremental improvements.

```
You are a senior React frontend engineer working with me on A+ Plane, a college grievance and campus support portal.

TECHNOLOGY
- React with Vite
- React Router for navigation
- Tailwind CSS for styling, where already configured
- Axios or the existing API service for backend requests
- Lucide React for icons, where already used
- Existing Express API and PostgreSQL-backed backend must be integrated through the existing service layer

FIRST: INSPECT, DO NOT GUESS
1. Inspect the current frontend folder, package.json, routes, components, API services, authentication hook/context,
2. Read the existing implementation before changing it.
3. Reuse existing components and conventions. Do not create duplicate pages, duplicate API clients, or a second authe
4. If a file or endpoint is missing, report that fact and propose the smallest compatible implementation.

PRODUCT GOAL
Build a polished, accessible, responsive student grievance portal called A+ Plane. The interface should feel like a c

REQUIRED FRONTEND AREAS
- Student login using the college-issued Student ID and password only.
- Student dashboard showing the authenticated student's name, course, semester, and section when those fields are ava
- Raise Grievance flow with the available categories and appropriate forms.
- My Complaints list and Complaint Details / tracking view.
- Admin dashboard with administrative actions permitted by the backend.
- Shared Navbar, Sidebar, ProtectedRoute, ComplaintCard, and StatusBadge components where appropriate.
- Hostel, Subjects, Bus, Her Voice, and Lost & Found grievance modules.
- Floating AI Campus Assistant that can guide users and call real backend APIs.
- Clear loading, empty, success, validation, and error states on every data-driven page.
- Responsive layouts for mobile, tablet, laptop, and desktop.

INTEGRATION RULES
- Use the existing API service instead of scattering raw fetch/axios calls throughout components.
- Keep authentication tokens and user state consistent with the existing auth implementation.
- Protect student and admin routes using verified authentication and role information from the backend.
- Do not hardcode real student records, complaint statuses, bus schedules, room availability, or admin data.
- Never expose secrets or database connection strings in frontend code.
- Never create an admin account or bypass authorization from the client.
- If the backend contract is unclear, inspect the backend routes and types before implementing the UI.

WORKING METHOD
1. Create a concise implementation plan, then continue with the code changes without stopping at the plan.
2. Work in small, coherent changes and preserve unrelated working code.
3. After each major change, run the relevant checks and fix errors that are actually observed.
4. Do not claim a test, build, API call, or deployment succeeded unless you ran it and saw the result.
5. At the end, report changed files, features completed, commands run, actual test/build results, and remaining blocl
```

3. Prompt — Login, Authentication UI, and Protected Routes

```
Improve the existing A+ Plane login and route-protection frontend.

- Inspect Login.jsx, ProtectedRoute.jsx, auth.js, api.js, App.jsx, and the backend authentication contract before edi
- Provide one student login method: college Student ID plus password. Do not add email login, social login, public st
- Provide the separate Admin login flow only if it is supported by the existing backend contract. Do not place admin
- Validate required fields, show clear inline errors, disable duplicate submission while loading, and provide a visib
- On successful login, use the existing auth service and navigate to the correct authorized dashboard based on verifi
- Do not trust a role selected by the browser or user. Do not treat hiding a button as authorization.
- Ensure ProtectedRoute handles loading, unauthenticated users, and role mismatch without redirect loops.
- Keep the screen responsive, keyboard accessible, and consistent with the A+ Plane design.
- Do not invent endpoints or response fields. If the existing backend contract does not support a requirement, report
- Run the available frontend checks and report their real results.
```

4. Prompt — Student Dashboard

```
Improve StudentDashboard.jsx for A+ Plane.

- Read the existing component, auth hook/context, route configuration, and student API endpoints first.
- Display the signed-in student's name, Student ID if authorized, course, semester, and section using authenticated d
- Add clear dashboard sections for raising a grievance, viewing submitted complaints, tracking complaint status, and
- Show useful summary cards only when the underlying values can be fetched reliably. Do not use invented complaint co
- Add loading skeletons or a clear loading indicator, empty states for first-time users, and actionable error messages
- Make every card or action navigate to a valid existing route.
- Keep layout responsive and accessible. Use reusable components where practical.
- Preserve existing working behavior and list any backend endpoints that are missing.
```

5. Prompt — Raise Grievance and Category Forms

```
Improve RaiseGrievance.jsx and the existing grievance module forms for A+ Plane.

Supported modules in the discussed frontend include:
- Hostel: HostelForm.jsx and hostelCategories.js
- Subjects: SubjectForm.jsx
- Bus: BusForm.jsx
- Her Voice: HerVoiceForm.jsx
- Lost & Found: LostFoundForm.jsx

Requirements:
1. Inspect each form and the backend complaint schema/routes before changing the field names or payload format.
2. Present the available grievance categories clearly and route each selection to its correct existing form.
3. Use controlled inputs and validate required fields on the client for usability, while relying on the backend for a
4. Prevent accidental repeated submissions; show loading, success, and failure states.
5. On a successful response, show the actual ticket/reference ID returned by the server and provide a working link to
6. Do not display a success message when the API request fails.
7. Preserve entered values when practical after a recoverable error.
8. Make forms keyboard accessible, mobile-friendly, and easy to understand.
9. Do not invent a form endpoint, category, ticket ID, or backend response. Confirm uncertain details by reading the
10. For sensitive Her Voice reports, use careful, non-judgmental wording and explain only privacy protections that th
11. For Lost & Found, ask only for information needed to describe and identify the item.
12. Report every changed file and any backend fields or endpoints that are missing.
```

6. Prompt — My Complaints and Complaint Tracking

```
Improve MyComplaints.jsx, ComplaintDetails.jsx, ComplaintCard.jsx, and StatusBadge.jsx.

- Read the existing complaint API routes and status enum before implementation.
- Fetch only the signed-in student's own complaints through authenticated endpoints.
- Display server-provided ticket ID, title, category, submission date, and current status where available.
- Provide useful loading, empty, error, and retry states.
- Let the user open a details page for a specific complaint and view the fields and status history actually returned
- Use a consistent status badge for the real statuses supported by the backend. Do not invent a status or imply that
- Handle a deleted, inaccessible, or unknown ticket with a clear not-found/unauthorized message.
- Do not expose another student's complaint by accepting arbitrary IDs without server-side ownership checks.
- Keep the list and details layout responsive and accessible.
- Verify that navigation paths and API calls match the actual project.
```

7. Prompt — Admin Dashboard, Navigation, and Shared UI

```
Improve AdminDashboard.jsx, Navbar.jsx, Sidebar.jsx, and NotFound.jsx for A+ Plane.

ADMIN DASHBOARD
- Inspect the actual admin endpoints and role checks first.
- Display complaint metrics and queues only when returned by the backend.
- Provide the administrative actions that the backend actually supports, such as reviewing a complaint, changing its
- Show confirmation and error states for actions that change records.
- Do not add fake charts, fake complaints, or client-only admin permissions.
- Do not expose student information beyond what an authorized admin is permitted to access.

NAVIGATION
- Keep the navigation consistent across the application.
- Show links appropriate to the authenticated user's verified role.
- Ensure every link points to a valid route and mobile navigation can be opened and closed by keyboard and touch.
- Display a clear logout action wired to the existing authentication service.
- Handle long labels and narrow screens without horizontal overflow.

SHARED UI
- Reuse ComplaintCard and StatusBadge where suitable.
- Ensure NotFound.jsx provides a clear message and a working way back to an allowed page.
- Use consistent spacing, focus states, contrast, icon sizing, and typography.
- Do not replace the existing design system wholesale unless necessary; make focused improvements.
- Run the frontend build and report actual results.
```

8. Prompt — Responsive Design and Accessibility Audit

```
Audit the existing A+ Plane frontend on viewport widths of approximately 320, 375, 430, 768, 1024, and 1440 pixels.

Check:
- No unintended horizontal overflow.
- Forms and buttons fit small screens and have comfortable touch targets.
- Tables, complaint lists, cards, sidebars, and dialogs adapt appropriately.
- The AI chatbot can be opened, used, and closed on mobile without covering essential controls.
- All inputs have labels; icon-only buttons have accessible names.
- Keyboard focus is visible; navigation and dialogs can be used without a mouse.
- Text contrast and status indicators remain understandable without relying only on color.
- Loading, empty, validation, success, and error states remain legible.
- Browser console errors and failed network requests are investigated rather than ignored.

Make targeted fixes in the existing files. Do not claim that all viewport sizes or accessibility requirements passed
```

9. Prompt — Upgrade the A+ Plane AI Campus Assistant

This prompt is based on the existing AIChatbot.jsx structure discussed for the project: a floating chat launcher, chat window, quick links, API call through aiApi.chat(messageText), and browser SpeechRecognition support. The AI assistant must inspect the current file and backend before editing.

```
Upgrade the existing AIChatbot.jsx into a reliable, polished A+ Plane Campus Assistant. Do not create a second chatbo

CURRENT IMPLEMENTATION TO PRESERVE WHERE USEFUL
- Floating chatbot launcher and chat window
- Message bubbles and automatic scroll-to-latest-message behavior
- Existing aiApi.chat(messageText) service integration
- Quick links and React Router navigation
- Browser SpeechRecognition / webkitSpeechRecognition support where available
- Existing Tailwind and Lucide React conventions

UI AND EXPERIENCE
- Create a refined, responsive chat panel that works on phones and desktops.
- Include a clear open/close control, readable conversation history, timestamps if useful, loading indicator, retry c
- Keep quick actions for raising a grievance and opening the Bus, Hostel, Her Voice, and Lost & Found sections only w
- Make the assistant's current status truthful. Do not always claim that it is online or AI-powered if the backend or
- Make sure targetPath from the API response is either safely validated against an explicit allowlist of internal rou
- Avoid duplicate message submission when the user presses Enter or clicks Send repeatedly.

VOICE
- Detect whether browser speech recognition is supported.
- Allow a clear language choice where supported, including English (India) and Hindi (India).
- Show listening, permission-denied, unsupported-browser, and recognition-error states.
- Stop recognition when the user ends listening, closes the chat, or the component unmounts.
- Do not claim that speech recognition works in every browser.
- Keep typed input usable when voice features are unavailable.

BACKEND AND PRIVACY
- Inspect aiApi, api.js, ai.routes.ts, ai.service.ts, authentication middleware, and available data endpoints.
- Use actual backend responses; do not invent bus timings, room availability, complaint status, or policy details.
- If the backend is unavailable, clearly label the response as a fallback and provide safe navigation links.
- Do not send another student's private complaint data to the model or reveal it in chat.
- Use authenticated identity where required and respect server-side authorization.
- Do not expose provider API keys or secrets in frontend code.
- For harassment or other sensitive reports, respond supportively and direct users to the appropriate official report

QUALITY
- Remove unused imports and unused state.
- Handle network failure, empty replies, malformed responses, and loading cleanup.
- Check for React warnings and runtime errors.
- Test the actual supported routes and API contract.
- At the end, list files changed, the actual AI/fallback mode, voice support limitations, and test results.
```

10. Prompt — Frontend and Backend Integration Audit

```
Perform an end-to-end integration audit of the existing A+ Plane frontend and backend.

Inspect frontend API services, auth state, routes, page components, Express routes, middleware, and the Prisma schema

Create a table with:
- Frontend feature/page
- Frontend file
- API method and path actually implemented
- Authentication/role required
- Request/response contract
- Current integration status
- Missing or mismatched items

Audit at least:
- Student login
- Admin login (if implemented by the backend)
- Student profile/dashboard
- Raise grievance for each available module
- My complaints
- Complaint details/status tracking
- Admin complaint management
- AI assistant
- Logout and protected routes

Then fix confirmed frontend/backend mismatches using the existing architecture. Do not invent endpoints or response

Run the available lint/build/test commands and record their actual outputs. Do not report success for checks that we
```

11. Prompt — Final Frontend Build and Bug-Fixing Pass

```
Run a final quality pass on the A+ Plane frontend.

1. Inspect package.json to identify the real available scripts.
2. Run the relevant lint, test, and production build commands.
3. Fix errors that can be reproduced from the command output or browser console.
4. Check imports, missing components, incorrect route paths, API base URL configuration, React key warnings, and unus
5. Check authentication/loading behavior and forms with both successful and failed API responses where testable.
6. Check mobile layout and the AI chatbot's open/close and voice behavior in a supported browser.
7. Do not remove features merely to make the build pass.
8. Do not fabricate API responses, test results, screenshots, or deployment status.
9. If credentials, a backend service, or an external AI provider are unavailable, document the blocker and the exact
10. Finish with a concise report: files changed, checks run, actual pass/fail results, unresolved issues, and next st
```

12. How This Documents Vibe Coding

Vibe coding is an iterative development style in which a developer describes the intended behavior in natural language, uses an AI coding assistant to generate or refine code, runs the application, and then improves the implementation using observed results. This prompt book documents that workflow for the A+ Plane frontend.

Suggested evidence to keep with the project

- Saved copies of the actual prompts entered into the coding assistant.
- A short record of which files were generated, modified, or reviewed after each prompt.
- Screenshots of the running interface, clearly labelled with the date and the page shown.
- Terminal output for the build, lint, and test commands that were actually run.
- A list of features that are complete, partially complete, or blocked by backend/API configuration.
- Git commits or a change log showing how the frontend evolved during development.

Important accuracy statement

> This PDF is a development prompt document. It should be presented as evidence of the intended AI-assisted workflow, not as proof by itself that all listed code was generated by AI, that every requirement is complete, or that the application passed all tests. Keep the original AI conversation or version-control history if a reviewer needs to verify the development process.

Quick final checklist

Check	What to confirm
Existing code reviewed	The AI inspected files before editing and reused the current architecture.
Pages and modules	Only pages and modules actually present are marked complete.
API integration	Network calls match implemented backend routes and response shapes.
Authentication	Student/admin access is enforced by backend authorization.
Chatbot	AI availability, fallback behavior, and speech support are described accurately.
Build and tests	Commands were run and their real outputs were saved.
Evidence	Prompts, screenshots, commits, and test outputs are retained where available.